```
    struct barrier
    {
        std::atomic<unsigned> count;
        std::atomic<unsigned> spaces;
        std::atomic<unsigned> generation;

        barrier(unsigned count_):
            count(count_),spaces(count_),generation(0)
        {}

        void wait()
        {
            unsigned const gen=generation.load();
            if(!--spaces)
            {
            spaces=count.load();
            ++generation;


            }
            else
            {
                while(generation.load()==gen)
                {
                    std::this_thread::yield();
                }
            }
    }

    void done_waiting()
    {
        --count;
        if(!--spaces)
        {
            spaces=count.load();
            ++generation;
        }
    }
};
```

The overall structure of this code is probably becoming familiar by now. You have a class with a function call operator (`process_element`) for doing the work ❶, which you run on a bunch of threads ❾ stored in a vector ❽ and which you also call from the main thread ❿. The key difference this time is that the number of threads is dependent on the number of items in the list rather than on